



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
Rai Technology University
(Estd. under Karnataka Act, No. 40 of 2013)

ASSIGNMENT

Subject Title: Deep Learning

Subject Code:OCS351T

SUBMITTED BY :

NAME: G ROSHAN ZAMEER

UEN NO : RTU24101CS035

PROGRAM NAME : B.TECH CSE

SEMESTER : 5TH SEM

SESSION : 2026-27

SUBMITTED TO :

FACULTY NAME : Mr.Satyam Sir

Department of Computer Science Engineering

College of Engineering

Rai Technology University Bengaluru-561204

ASSIGNMENT

Simple Neural Network

INTRODUCTION :-

Hand written digit recognition is a common image classification problem in machine learning. The objective is to automatically identify a hand written digit from an input image.

The MNIST dataset is widely used for evaluating image classification algorithms. It contains grayscale images of handwritten digits from 0 to 9.

Problem Statement :-

To develop a neural network using TensorFlow/Keras that can classify handwritten digit images into one of ten classes, from 0 to 9.

Objectives :-

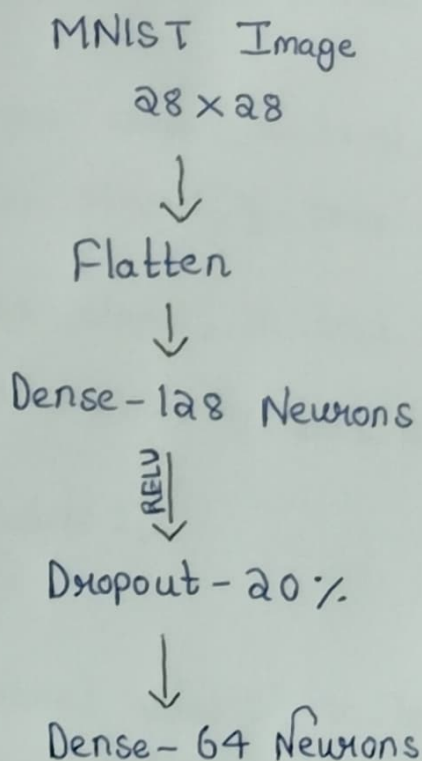
1. Load the MNIST dataset
2. Explore the dataset
3. Preprocess the images
4. Build a neural network.
5. Train a model
6. Evaluate it's accuracy.

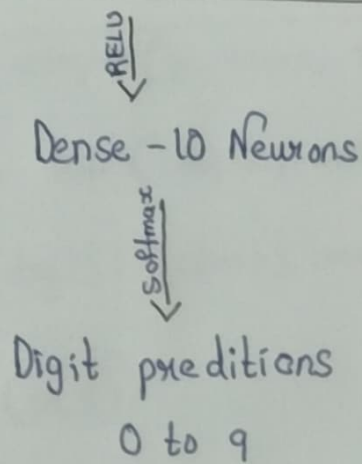
7. Visualize training performance
8. Test five images
9. perform an architecture experiment.

Methodology :-

1. Load the MNIST dataset
2. Explore the dataset & display sample images.
3. Normalize pixel values.
4. Flatten each 28×28 image.
5. Pass the flattened image through the hidden layers.
6. Use the output layer containing 10 neurons.
7. Train the network.
8. Evaluate it on test dataset.

Model Architecture :-





CODE :-

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense, Dropout

Print("TensorFlow Version :", tf.__version__)

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()

Print("Training images shape :", X_train.shape)
Print("Training labels shape :", y_train.shape)

Print("Testing images shape :", X_test.shape)
Print("Testing labels shape :", y_test.shape)

Print("First 20 labels :")
Print(y_train[:20])

Print("\nMinimum pixel value :", X_train.min())
Print("Maximum pixel value :", X_train.max())
```

```
unique, counts = np.unique(y_train, return_counts=True)
for digit, count in zip(unique, counts):
    print(f"Digit {digit}: {count} images")
```

```
plt.figure(figsize=(10, 5))
```

```
for i in range(10):
```

```
    plt.subplot(2, 5, i+1)
```

```
    plt.imshow(X_train[i], cmap='gray')
```

```
    plt.title(f"Label: {y_train[i]}")
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
X_train = X_train.astype("float32")/255.0
```

```
X_test = X_test.astype("float32")/255.0
```

```
print("Minimum value after normalization:", X_train.min())
```

```
print("Maximum value after normalization:", X_train.max())
```

```
model = Sequential([
```

```
    Input(shape=(28, 28)),
```

```
    Flatten(),
```

```
    Dense(128, activation='relu'),
```

```
    Dropout(0.2),
```

```
    Dense(64, activation='relu'),
```

```
    Dense(10, activation='softmax').
```

```
])
```

```
model.summary()
```

```
model.compile(  
    optimize='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)
```

```
print("Model compiled successfully.")
```

```
history = model.fit(  
    x_train,  
    y_train,  
    epochs=10,  
    batch_size=128,  
    validate_split=0.1,  
    verbose=1  
)
```

```
)  
test_loss, test_accuracy = model.evaluate(  
    x_test,  
    y_test,  
    verbose=0  
)
```

```
print("Test Loss:", test_loss)
```

```
print("Test Accuracy:", test_accuracy)
```

```
print("Test Accuracy (%):", test_accuracy * 100)
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
```

```
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
```

```
plt.title('Training & Validation Accuracy')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.title('Training & Validation Loss')
```

```
plt.xlabel('Epoch')
```

```
plt.ylabel('Loss')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
predictions = model.predict(X_test)
```

```
predicted_labels = np.argmax(predictions, axis=1)
```

```
print("First 20 predicted labels:")
```

```
print(predicted_labels[:20])
```

```
sample_indices = [0, 1, 2, 3, 4]
```

```
plt.figure(figsize=(12, 3))
```

```
for i, index in enumerate(sample_indices):
```

```
    image = X_test[index]
```

actual = y_test[index]

predicted = predicted_labels[index]

plt.subplot(1, 5, i+1)

plt.imshow(image, cmap='gray')

plt.title(f"Actual : {actual} \n Predicted: {predicted}")

plt.axis('off')

plt.tight_layout()

plt.show()

Print("Actual vs Predicted labels")

Print("-" * 35)

for index in sample_indices:

 actual = y_test[index]

 predicted = predicted_labels[index]

 result = "Correct" if actual == predicted else "Incorrect"

 Print (

 f"image {index + 1}:"

 f"Actual = {actual},"

 f"Predicted = {predicted},"

 f"Result = {result}"

)

```
experiment_model = Sequential([  
    Input(shape=(28, 28)),  
    Flatten(),  
    Dense(256, activation='relu'),  
    Dropout(0.2),  
    Dense(128, activation='relu'),  
    Dense(10, activation='softmax')  
])
```

```
experiment_model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metric=['accuracy']  
)
```

```
experiment_history = experiment_model.fit(  
    x_train,  
    y_train,  
    epochs=10,  
    batch_size=128,  
    validation_split=0.1,  
    verbose=1  
)
```

```
experiment_loss, experiment_accuracy = experiment_model.evaluate(  
    x_test,  
    y_test,  
    verbose=0
```

```
)  
print("Experimental Model Test Loss:", experiment_loss)
```

```
print (
```

```
    "Experimental Model Test Accuracy:",  
    experiment_accuracy
```

```
)
```

```
print (
```

```
    "Experimental Model Test Accuracy (%):",  
    experiment_accuracy * 100
```

```
)
```

Conclusion :-

The MNIST handwritten digit classification Project successfully demonstrated the use of Tensor Flow/keras to build image classification.